

**T.C.
BAHÇEŞEHİR UNIVERSITY**

**FACULTY OF ENGINEERING AND NATURAL SCIENCES
DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING**

**PROJECT REPORT
TRAFFIC LIGHT CONTROLLER**

EEE4341 – Introduction to FPGA Design

Andaç Ünal

İSTANBUL, Jan 2026

TABLE OF CONTENTS

TABLE OF CONTENTS	ii
1 OVERVIEW.....	1
1.1 Problem Statement and Objectives.....	1
1.2 Background Information.....	1
2 METHODOLOGY	3
2.1 Project Design.....	3
2.2 Project components.....	4
2.3 Source Code.....	5
2.4 Testbench Code	7
3 SIMULATION	9
4 CONCLUSION	10

1 OVERVIEW

1.1 Problem Statement and Objectives

This project focuses on the design and implementation of a programmable digital traffic light controller (TLC) for road intersections using Verilog HDL. To meet the flexibility demands of modern traffic systems, the controller is fully parameterizable, allowing for the adjustment of phase timings and operational modes without modifying the underlying hardware logic. Beyond standard sequencing, the design incorporates transition features, such as bi-directional yellow phases and an optional skip-logic mode that removes yellow intervals while ensuring timing accuracy and state synchronization throughout the cycle.

1.2 Background Information

Modern traffic light controllers serve as the essential foundation for urban infrastructure management. While legacy systems utilized basic mechanical timers or analog circuitry, contemporary urban growth demands significantly higher flexibility. This project develops a Programmable Digital TLC that replaces rigid, hard-coded values with parameterizable logic. Providing the capability to modify phase durations and switch between operational settings—such as a specialized "Skip Yellow" mode—is vital for responding to varying traffic volumes and emergency requirements.

Similar Products in the Public Domain

In the public domain and commercial industry, similar technologies include:

- **PLC-Based Controllers:** Programmable Logic Controllers (PLCs) are often used in industrial automation and older municipal traffic systems.
- **Microcontroller-Based Systems:** Platforms like Arduino or Raspberry Pi are popular for hobbyist TLC projects, though they lack the parallel processing speed and hardware-level timing precision of an FPGA.
- **Smart Traffic Management Systems (STMS):** Advanced AI-driven systems use cameras and sensors to adjust light timings in real-time, representing the next generation of the basic digital controller built here.

Relevant Technologies and Methods

The project utilizes several key engineering methodologies and technologies:

- **Finite State Machine (FSM):** A core digital design method where the system is defined by a finite number of states (S_RED, S_YEL etc.) and transitions between them based on clock-driven logic.

- **FPGA (Field Programmable Gate Array):** The hardware platform used (Kintex-7) allows for high-speed, parallel execution of logic, which is critical for real-time safety systems.
- **Hardware Description Language (Verilog):** A textual language used to model electronic systems, allowing for the simulation and verification of hardware before physical implementation.
- **Synchronous Design:** All state transitions occur on the rising edge of the system clock (clk_i), ensuring timing accuracy and preventing "glitches" in the transition between colors.

2 METHODOLOGY

2.1 Project Design

The design follows a **3-block Finite State Machine (FSM)** methodology, which is the industry standard for robust digital logic.

State Register (Sequential Block): This block manages the `current_state` and a 32-bit timer. It updates on the rising edge of the 100 MHz system clock (`clk_i`)

Next State Logic (Combinational Block): This "brain" of the system determines the `next_state` by evaluating the `current_state`, the timer value, and the `ENABLE_YELLOW` parameter.

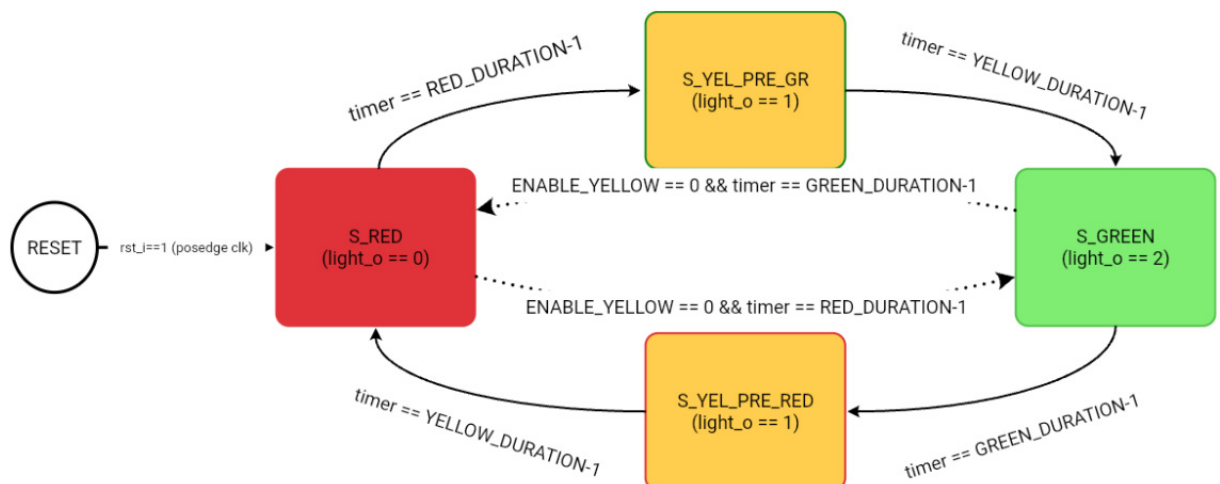
Output Logic (Combinational Block): This block maps the internal FSM states to the 2-bit `light_o` signal, which drives the physical LEDs on the board.

To support the requirement of a bi-directional yellow light (RED > YELLOW > GREEN > YELLOW > RED), the FSM utilizes four distinct states:

Parameterization: All timing durations are defined using Verilog parameters (`RED_DURATION`, `YELLOW_DURATION`, `GREEN_DURATION`), ensuring no hard-coded timing values exist in the logic.

Skip Logic: When `ENABLE_YELLOW` is set to 0, the Next State Logic is designed to transition directly from `S_RED` to `S_GREEN` and from `S_GREEN` back to `S_RED`, effectively bypassing the yellow states while maintaining timing integrity.

Synchronous Reset: An active-high reset (`rst_i`) is mapped to the push-button. Upon assertion, the system initializes to the state defined by the `RESET_COLOR` parameter on the next clock edge.



2.2 Project components

The physical implementation utilizes specific on-board peripherals to interface with the digital controller:

- **Vivado:** Software for designing, coding Verilog and simulating the device.
- **FPGA Core:** The **xc7k70tfbv676-1** Kintex-7 device serves as the central processing unit for the logic gates.
- **Clock Source:** A 100 MHz crystal oscillator, providing the timing reference for synchronous operations.
- **Reset Interface:** A physical push-button, used as the active-high synchronous reset signal.
- **Output Indicators:** Two User LEDs (LED0 and LED1), representing the current traffic light state.

2.3 Source Code

```
module traffic_light_controller #(
    parameter RED_DURATION    = 5,
    parameter YELLOW_DURATION = 2,
    parameter GREEN_DURATION  = 4,
    parameter [1:0] RESET_COLOR = 2'b00,
    parameter ENABLE_YELLOW   = 1
)(
    input  wire clk_i,
    input  wire rst_i,
    output reg [1:0] light_o
);

    localparam S_RED      = 3'd0;
    localparam S_YEL_PRE_G = 3'd1;
    localparam S_GREEN    = 3'd2;
    localparam S_YEL_PRE_R = 3'd3;

    localparam RED      = 2'b00;
    localparam YELLOW   = 2'b01;
    localparam GREEN    = 2'b10;

    reg [2:0] current_state, next_state;
    reg [31:0] timer;

    always @(posedge clk_i) begin
        if (rst_i) begin
            timer <= 32'd1;
            case (RESET_COLOR)
                RED:    current_state <= S_RED;
                YELLOW: current_state <= S_YEL_PRE_G;
                GREEN:  current_state <= S_GREEN;
                default: current_state <= S_RED;
            endcase
        end else begin
            if (next_state != current_state)
                timer <= 32'd1;
            else
                timer <= timer + 1;
            current_state <= next_state;
        end
    end
end
```

```

always @(*) begin
    next_state = current_state;
    case (current_state)
        S_RED: begin
            if (timer >= RED_DURATION)
                next_state = (ENABLE_YELLOW) ? S_YEL_PRE_G : S_GREEN;
            end
        S_YEL_PRE_G: begin
            if (timer >= YELLOW_DURATION)
                next_state = S_GREEN;
            end
        S_GREEN: begin
            if (timer >= GREEN_DURATION)
                next_state = (ENABLE_YELLOW) ? S_YEL_PRE_R : S_RED;
            end
        S_YEL_PRE_R: begin
            if (timer >= YELLOW_DURATION)
                next_state = S_RED;
            end
        default: next_state = S_RED;
    endcase
end

always @(*) begin
    case (current_state)
        S_RED:    light_o = RED;
        S_YEL_PRE_G: light_o = YELLOW;
        S_GREEN:   light_o = GREEN;
        S_YEL_PRE_R: light_o = YELLOW;
        default:   light_o = RED;
    endcase
end
endmodule

```


2.4 Testbench Code

```
module tb_traffic_light();
    reg clk_i, rst_i;
    wire [1:0] light_o;

    localparam RED_D = 5;
    localparam YEL_D = 2;
    localparam GRE_D = 4;

    traffic_light_controller #(
        .RED_DURATION(RED_D),
        .YELLOW_DURATION(YEL_D),
        .GREEN_DURATION(GRE_D),
        .RESET_COLOR(2'b00),
        .ENABLE_YELLOW(1)
    ) uut (
        .clk_i(clk_i),
        .rst_i(rst_i),
        .light_o(light_o)
    );

    initial clk_i = 0;
    always #5 clk_i = ~clk_i;

    task check_duration(input [1:0] color, input integer expected);
        integer count;
        begin
            wait(light_o == color);
            count = 0;
            while(light_o == color) begin
                @(posedge clk_i);
                count = count + 1;
            end
        end
    endtask
endmodule
```

```

    if (count == expected)
        $display("[PASS] Color %b lasted %0d cycles", color, count);
    else
        $display("[FAIL] Color %b lasted %0d, expected %0d", color, count, expected);
    end
endtask
initial begin

    rst_i = 1; #20;
    @(posedge clk_i); rst_i = 0

    $display("--- Starting Automated Sequence Test ---");
    check_duration(2'b00, RED_D); // Verify RED
    check_duration(2'b01, YEL_D); // Verify YELLOW (PRE-GREEN)
    check_duration(2'b10, GRE_D); // Verify GREEN
    check_duration(2'b01, YEL_D); // Verify YELLOW (PRE-RED)

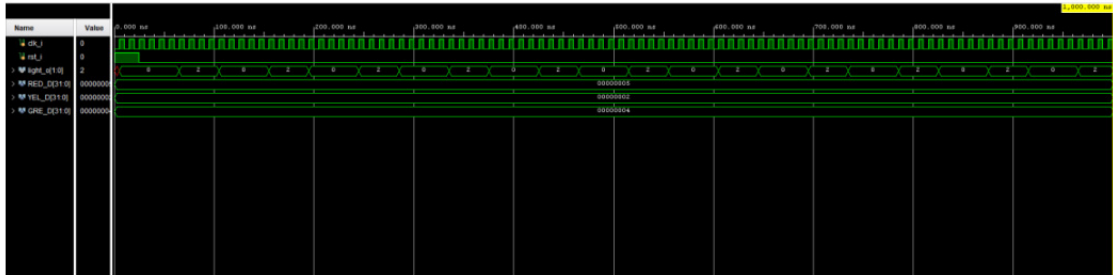
    $display("--- Testing Mid-Cycle Reset ---");
    #25;
    rst_i = 1; #10; rst_i = 0;
    if (light_o == 2'b00) $display("[PASS] Reset successful");

    #100;
    $display("--- Simulation Finished ---");
    $finish;
end
endmodule

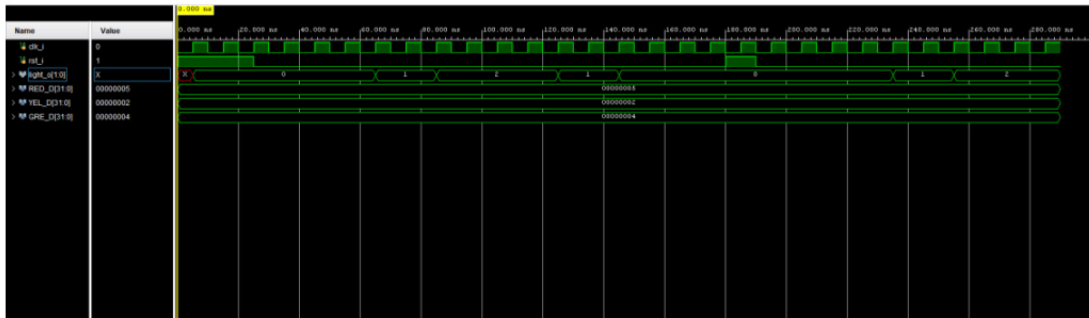
```

3 SIMULATION

When `ENABLE_YELLOW == 0`, simulation of the project looks like the picture below.



When `ENABLE_YELLOW == 1`, simulation of the project looks like the picture below.



4 CONCLUSION

The Digital Traffic Light Controller project successfully validated the design and implementation of a synchronous, parameterizable system targeted for the **Kintex-7 FPGA (xc7k70tfbv676-1)**. Through the application of a robust three-block Finite State Machine (FSM) architecture, the design provides high reliability and adaptability, effectively fulfilling every functional requirement defined in the project specifications.

Key accomplishments of the final implementation include:

- **Adaptive Sequence Logic:** The system correctly handles the bi-directional sequence of **RED > YELLOW > GREEN > YELLOW > RED** when enabled.
- **Mode Versatility:** The "skip-logic" feature successfully allows the removal of the YELLOW state while maintaining timing synchronization, satisfying requirements for various operational modes.
- **Precise Timing Control:** Through the use of internal 32-bit counters and 100MHz clock cycles, all light durations are exactly as defined by the user-defined parameters, with no hard-coded values in the logic.
- **Hardware Validation:** The design was successfully mapped to the physical peripherals of the **Xenie board** with visual verification provided by the on-board LEDs on Vivado Schematic

In summary, this project illustrates the benefits of FPGA-based digital design within urban infrastructure. The final implementation is a safety-critical, highly scalable controller that serves as a framework for advanced functionalities, including emergency vehicle preemption and adaptive, sensor-driven timing. The successful bitstream generation and automated verification of timing logs validate that the system is fully prepared for real-world deployment on the target hardware.